

Documentazione progetto TAW

Backend - Database

Giovanni Quacchia - 900676

Settembre 2025

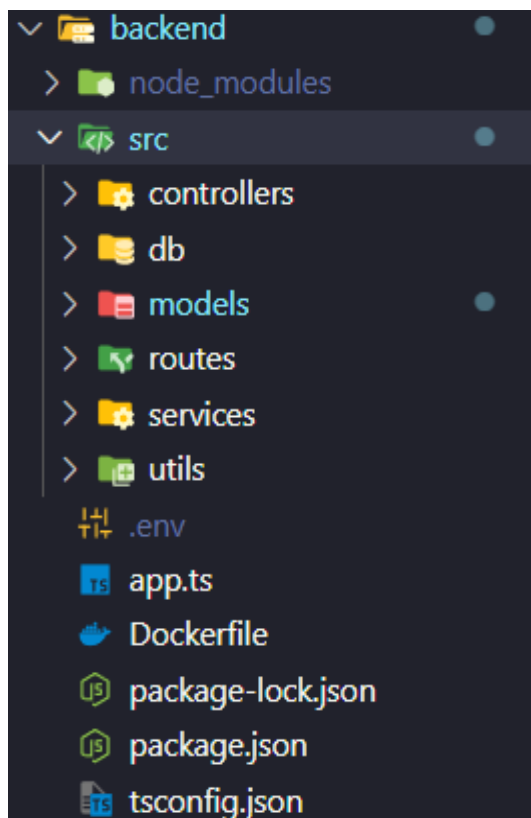
Introduzione.....	3
Backend.....	3
Requisiti.....	4
Endpoints.....	5
Airport.....	5
Route.....	7
Airplane.....	10
Funzionamento dei posti.....	10
Airline.....	13
User.....	19
Flight.....	23
Ticket.....	28
Purchase.....	31
Passenger.....	34
Itinerary.....	37
Authentication.....	38
JWT.....	38

Introduzione

Il progetto per la ricerca dei voli e l'acquisto dei biglietti è strutturato in frontend, realizzato come SPA in Angular, backend in Express JS e il database NOSQL MongoDB.

Backend

Il backend è strutturato in un file **app.ts** in cui sono specificate gli endpoints principali e viene stabilita la connessione al database.



Nella cartella **src** sono presenti ulteriori sotto cartelle

Models: Definizione delle interfacce e schemi per il DB

Routes: Endpoints specifiche delle interfacce per operazioni CRUD

Controllers: Gestione degli errori e validazione dell'input dell'utente

Services: Gestione delle query inviate al DB

DB: Dati di inizializzazione per il DB e logica delle query

Utils: Logica per autenticazione e altre operazioni utili

Requisiti

1. Gestione utenti
 - a. [Registrazione](#)
 - b. [Registrazione compagnie aeree](#)
 - c. [Eliminazione utenti](#) (Un utente può eliminare solo il proprio account)
2. Gestione voli
 - a. [Creazione rotte](#)
 - b. [Creazione aeroplani](#)
 - c. [Creazione voli](#)
 - d. [Aggiornare i prezzi dei biglietti](#)
 - e. [Statistiche sui voli](#) (/airlines/:id/flights)
 - f. [Statistiche sulle rotte](#) (/airlines/:id/routes)

3. Ricerca voli

- a. [Ricerca dei voli](#) (fino a 2 scali) e [acquisto dei biglietti](#)

Un utente effettua un acquisto specificando l'ID del biglietto e il numero di biglietti da acquistare

Successivamente crea i passeggeri specificando nome, cognome, CF o passaporto, posto, extra e ID dell'acquisto

4. Biglietti e posti

- a. [Acquisto dei biglietti](#)

Un utente può acquistare dei biglietti per dei passeggeri che non sono necessariamente loggati

- b. [Selezione dei posti](#)

Nel momento in cui un passeggero viene creato, si sceglie il posto ed eventuali extra. Il backend verifica che il posto esista nell'aeroplano relativo al volo e che sia libero.

Endpoints

Airport	
name	string
city	string
code	IATA code, /^[A-Z]{3}\$/
country	string

/airports	GET
-----------	-----

Params: ?q

GET	backend:3000/api/airports?q=italy
<pre>[{ "_id": "6510a1f43b7e41b94cfe4d12", "__v": 0, "city": "Venice", "code": "VCE", "country": "Italy", "name": "Venice Marco Polo Airport" }, { "_id": "6510a1f43b7e41b94cfe4d13", "__v": 0, "city": "Rome", "code": "FCO", "country": "Italy", "name": "Rome Fiumicino Airport" }]</pre>	

Ritorna una lista di tutti gli aeroporti. Il parametro q filtra su tutti i campi di ogni documento operando come un LIKE

/airports/:id	GET
---------------	-----

Ritorna un aeroporto identificato da :id

/airports	POST
-----------	------

Crea un nuovo aeroporto, specificando tutti i campi (name, city, code, country)

Solo admin e compagnie aeree

/airports/:id	DELETE
---------------	--------

Elimina un aeroporto identificato da :id

Solo admin

/airports/:id	PUT
---------------	-----

Aggiorna un aeroporto identificato da :id. Possono essere specificati tutti i campi (name, city, code, country)

Solo admin

Route	
from	ID ['Airport']
to	ID ['Airport']

/routes	GET
---------	-----

Params: ? from, to

```

GET backend:3000/api/routes?from=italy&to=jfk

{
  "_id": "6520a1f43b7e41b94cfe4d22",
  "__v": 0,
  "from": {
    "_id": "6510a1f43b7e41b94cfe4d12",
    "__v": 0,
    "city": "Venice",
    "code": "VCE",
    "country": "Italy",
    "name": "Venice Marco Polo Airport"
  },
  "to": {
    "_id": "6510a1f43b7e41b94cfe4d14",
    "__v": 0,
    "city": "New York",
    "code": "JFK",
    "country": "USA",
    "name": "John F. Kennedy International Airport"
  }
}

```

Ritorna una lista di tutte le rotte disponibili da un aeroporto a un altro. I parametri from e to filtrano su tutti i campi dell'aeroporto, come il parametro q in /airports

/routes/:id	GET
-------------	-----

Ritorna una rotta identificata da :id

/routes	POST
---------	------

POST backend:3000/api/routes

Params Authorization **Headers (10)** Body Scripts Settings

Headers 9 hidden

Key	Value	Description
☒ authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1...	
☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL		
☒ from	6510a1f43b7e41b94cfe4d12	VCE
☒ to	6510a1f43b7e41b94cfe4d15	LAX

```
{
  "from": "6510a1f43b7e41b94cfe4d12",
  "to": "6510a1f43b7e41b94cfe4d15",
  "_id": "68cbfd223ccd442568a77e47",
  "__v": 0
}
```

Crea una nuova rotta, specificando i campi (from, to). Viene inoltre controllato che from != to

Solo admin e compagnie aeree

/routes/:id	DELETE
-------------	--------

Elimina una rotta identificata da :id

Solo admin

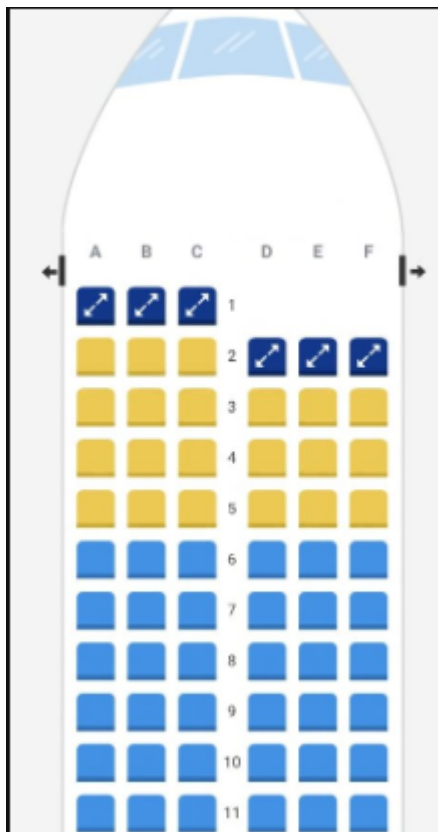
/routes/:id	PUT
-------------	-----

Aggiorna una rotta identificata da :id. Possono essere specificati i campi (from, to). Viene inoltre controllato che from != to

Solo admin

Airplane	
code	number
model	string
airline	ID ['Airline']
rows	number
letters	number
route?	ID ['Route']
startDate?	date
endDate?	date

Funzionamento dei posti



Ogni aereo ha i campi

- letters (numero di colonne)
- rows (numero di righe)

e consentono di determinare i posti validi sull'aereo.

Nell'immagine a fianco (anche se l'aereo non è completo) ipotizziamo di avere

- letters = 6, ovvero lettere da A ad F
- rows = 11

Quindi i posti validi saranno: A1, A2, A3, ..., A11, B1, ..., B11, ..., F11

/airplanes	GET
------------	-----

```

GET backend:3000/api/airplanes

[
  {
    "_id": "6540a1f43b7e41b94cfe6d01",
    "__v": 0,
    "airline": {
      "_id": "650f0a4f3b7e41b94cfe4d",
      "code": "FR",
      "name": "Ryanair",
      "PIVA": "IE123456789",
      "logo": "https://cdn.worldvecto
    },
    "code": 1,
    "letters": 6,
    "model": "Airbus A320neo",
    "rows": 30
  }
]

```

Ritorna una lista di tutti gli aeroplani.

/airplanes/:id	GET
----------------	-----

Ritorna un aeroplano identificato da :id

/airplanes	POST
------------	------

Crea un nuovo aeroplano, specificando tutti i campi. La rotta assegnata e il periodo (startDate, endDate) sono campi facoltativi, ma la presenza della rotta determina anche la necessità del periodo

Solo admin e compagnie aeree

/airplanes/:id	DELETE
----------------	--------

Elimina un aeroplano identificato da :id

Solo admin

/airplanes/:id	PUT
----------------	-----

Aggiorna un aeroplano identificato da :id.

Solo admin

Airline	
mail	string ['mail']
salt?	string
digest?	string
isFirstLogin	boolean
code	string
name	string
PIVA	string
logo?	string

/airlines/sessions	POST
--------------------	------

Invia una richiesta di login. Sono richieste mail, password e newPassword al primo accesso. Il server restituisce un JWT

POST	backend:3000/api/airlines/sessions
------	------------------------------------

Key	Value
☒ mail	contact@airfrance.com
☒ password	password
Key	Value
<pre>{ "type": "Input error", "msg": "Please provide a new password on first login" }</pre>	

☒	mail	contact@airfrance.com
☒	password	password
☒	newPassword	test

```

{
  "msg": "Password updated",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1MGYwYTRmM2I3ZTQxYjk0Y2ZlNGQxNiIsIm1hYWwiOiJjb250YWN0QGpZyYw5jZS5jb2MDAwMSwiZXhwIjoxNzU4ODA0ODAxZQ._gCSzNSJ4Fd9-_ -xRhPVLVvMrIzRJBxD1U02uTSc8zo",
  "expireDays": 7
}

```

/airlines	GET
-----------	-----

Params: name

Ritorna una lista di tutte le compagnie aeree. Il parametro name ricerca una compagnia sul campo name.

GET	backend:3000/api/airlines?name=emirates
-----	---

[	<pre> { "_id": "650f0a4f3b7e41b94cfe4d18", "code": "EK", "name": "Emirates", "PIVA": "AE987654321", "logo": "https://upload.wikimedia.org/wikipedia/commons/d/d8/Emirates_logo.svg" } </pre>	]
---	--	---

Solo l' admin può vedere tutte le informazioni (mail, salt, digest)

[	<pre> { "_id": "650f0a4f3b7e41b94cfe4d18", "mail": "contact@emirates.com", "isFirstLogin": true, "code": "EK", "name": "Emirates", "PIVA": "AE987654321", "logo": "https://upload.wikimedia.org/wikipedia/commons/d/d8/Emirates_logo.svg", "salt": "b0870a8d750d8fa7fb28056469e935a4", "digest": "ec02054e169d7d4944bd95519050b26bbc3d6a0a03e744f45b7364419a36190fb2183ae9a66cf7ec070c8f85e876919da6b1d9b02d1a91bf91ac630b6", "__v": 0 } </pre>	]
---	---	---

/airlines/:id	GET
---------------	-----

Ritorna la compagnia aerea identificata da :id

Solo l' admin può vedere tutte le informazioni (mail, salt, digest)

La compagnia aerea può vedere anche la sua mail



/airlines/:airlineId/airplanes	GET
--------------------------------	-----

Ritorna gli aeroplani della compagnia aerea identificata da :airlineId

/airlines/:airlineId/flights	GET
------------------------------	-----

Params: statistics

statistics: parametro booleano per calcolare numero di passeggeri e ricavato totale per ciascun volo (numPassengers e totRevenue)

Ritorna i voli della compagnia aerea identificata da :airlineId

```
GET backend:3000/api/airlines/650f0a4f3b7e41b94cfe4d12/flights?statistics=true

authorization eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp... ryanair

[
  {
    "_id": "6530a1f43b7e41b94cfe5d05",
    "__v": 0,
    "airline": { ...
  },
  "airplane": { ...
  },
  "arrival": "2025-09-16T18:30:00.000Z",
  "code": "FR505",
  "departure": "2025-09-16T09:30:00.000Z",
  "duration": 540,
  "route": { ...
  },
  "numPassengers": 1,
  "totRevenue": 2200
  },
  {
    ...
  }
]
```

/airlines/:airlineId/tickets	GET
------------------------------	-----

Ritorna i biglietti della compagnia aerea identificata da :airlineId

/airlines/:airlineId/routes	GET
-----------------------------	-----

Params: sortBy, order

sortBy: numPassengers

Ritorna le rotte servite dalla compagnia aerea identificata da :airlineId.

Solo admin e la compagnia aerea specifica ricevono anche numPassengers con params: sortBy=numPassengers&order=desc di default.

Il primo risultato sarà quindi la rotta “più richiesta”

```
GET backend:3000/api/airlines/650f0a4f3b7e41b94cfe4d12/routes
```


☒ authorization eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp... ryanair

```
[
  {
    "_id": "6520a1f43b7e41b94cfe4d22",
    "__v": 0,
    "from": { ...
  },
  "to": { ...
  },
  "numPassengers": 2
},
{
  "_id": "6520a1f43b7e41b94cfe4d2b",
  "__v": 0,
  "from": { ...
  },
  "to": { ...
  },
  "numPassengers": 1
}
]
```

/airlines	POST
-----------	------

Crea una nuova compagnia aerea, specificando i campi: code, mail, PIVA, name e un logo facoltativo

Solo admin

/airlines/:id	DELETE
---------------	--------

Elimina una compagnia aerea identificata da :id

Solo admin

/airlines/:id	PUT
---------------	-----

Aggiorna una compagnia aerea identificata da :id, specificando i campi code, mail, PIVA, name, logo, password, newPassword

Solo admin e compagnia aerea specifica

Solo l'admin può aggiornare l'email

User	
mail	string ['mail']
roles?	string[]
salt?	string
digest?	string
balance?	number

/users/sessions	POST
-----------------	------

Invia una richiesta di login. Sono richieste mail, password.
Il server restituisce un JWT

```

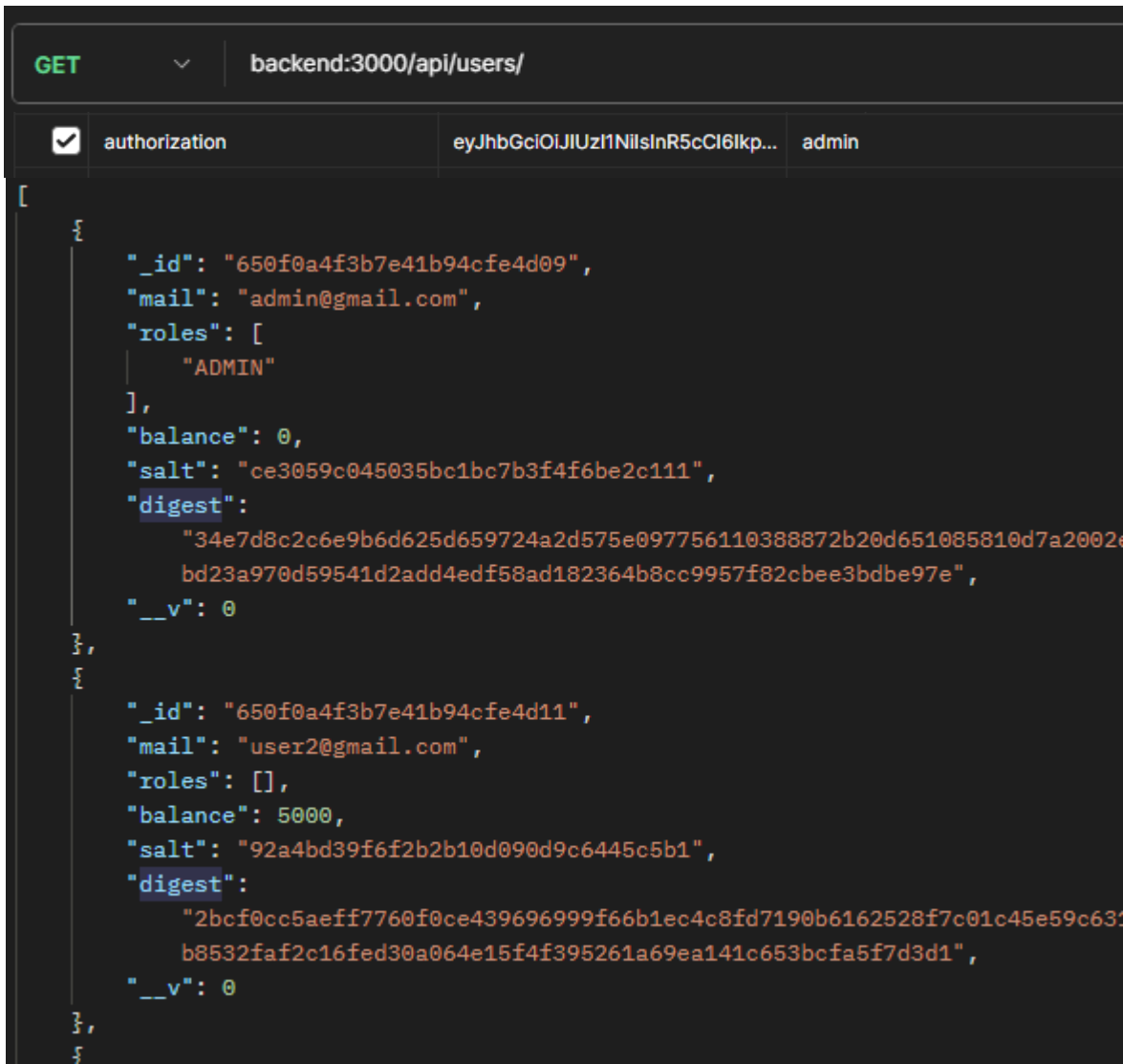
POST backend:3000/api/users/sessions
{
  "mail": "user@gmail.com",
  "password": "user"
}
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1MGYwYTRmM2I3ZTQxYjk0Y2ZlNGQxMCI6Im1haWwiOiJ1c2VyQGdtYWlsLmNvbSI6Im1zQWRtaW4iOmZhbHN1LCJpYXQiOiJlE3NTgyMDM4ODcsImV4cCI6MTc1ODgwODY4N30uAyLnM1uTlrFy9z1-xmu9HSQsxmChMmS4-rnaoVGCmC4",
  "expireDays": 7
}

```

/users	GET
--------	-----

Params: mail

Ritorna una lista di tutti gli utenti



Solo admin

/users/:id	GET
------------	-----

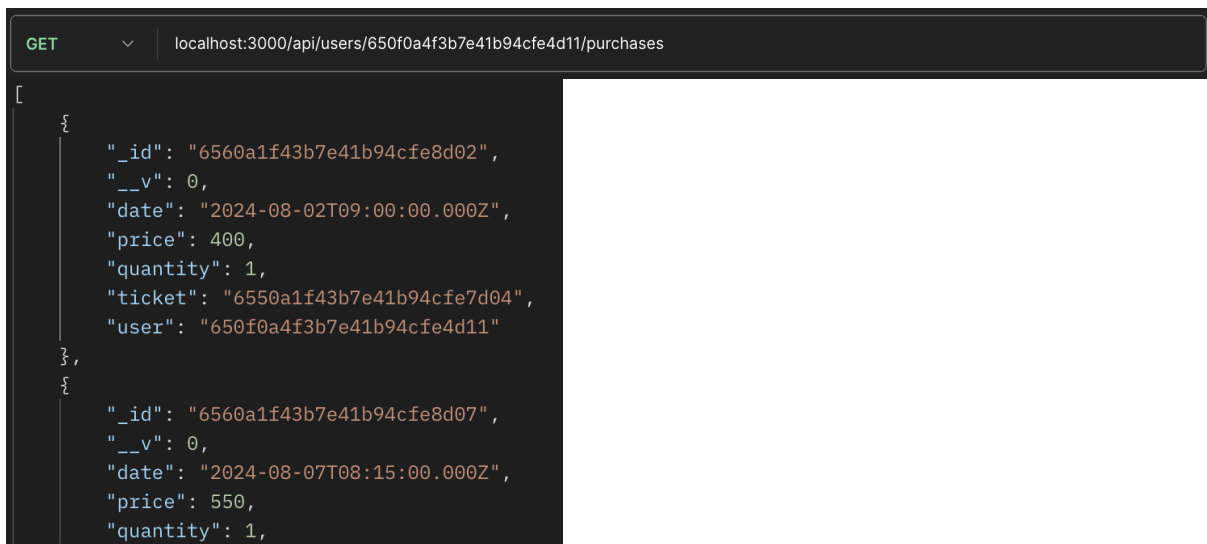
Ritorna un utente identificato da :id

Solo admin e utente specifico. L'utente vede solo mail e balance



/users/:userId/purchases	GET
--------------------------	-----

Ritorna una lista degli acquisti effettuati da un utente identificato da :userId



/users	POST
--------	------

Crea un nuovo utente, specificando mail e password. Il server ritorna un JWT.

/users/:id	DELETE
------------	--------

Elimina un utente identificato da :id

Solo admin e utente specifico

/users/:id	PUT
------------	-----

Aggiorna un utente identificato da :id, specificando mail, password, newPassword e balance.

Solo l'admin può aggiornare il saldo.

Solo admin e utente specifico

Flight	
code	string
departure	date
arrival	date
duration	number
route	ID ['Route']
airline	ID ['Airline']
airplane	ID ['Airplane']

/flights	GET
----------	-----

Params: from, to, fromDate, toDate, airline, airplane, code, sortBy, order,

sortBy: duration, departure, arrival

order: asc, desc

GET	▼	backend:3000/api/flights?from=italy&to=usa&airline=ryanair
-----	---	--

```
[
  {
    "_id": "6530a1f43b7e41b94cfe5d05",
    "__v": 0,
    "airline": {
      "_id": "650f0a4f3b7e41b94cfe4d12",
      "code": "FR",
      "name": "Ryanair",
      "PIVA": "IE123456789",
      "logo": "https://cdn.worldvectorlogo.com/logos/ryanair-1.svg"
    },
    "airplane": { ...
  },
  "arrival": "2025-09-16T18:30:00.000Z",
  "code": "FR505",
  "departure": "2025-09-16T09:30:00.000Z",
  "duration": 540,
  "route": {
    "_id": "6520a1f43b7e41b94cfe4d22",
    "__v": 0,
    "from": {
      "_id": "6510a1f43b7e41b94cfe4d12",
      "__v": 0,
      "city": "Venice",
      "code": "VCE",
      "country": "Italy",
      "name": "Venice Marco Polo Airport"
    },
    "to": {
      "_id": "6510a1f43b7e41b94cfe4d14",
      "__v": 0,
      "city": "New York",
      "code": "JFK",
      "country": "USA",
      "name": "John F. Kennedy International Airport"
    }
  }
},
5
```


Ritorna una lista di tutti i voli.

/flights/:id	GET
--------------	-----

Ritorna un volo identificato da :id

/flights/:flightId/tickets	GET
----------------------------	-----

Ritorna i biglietti relativi al volo :flightId

GET	backend:3000/api/flights/6530a1f43b7e41b94cfe5d05/tickets
<pre>[{ "_id": "6550a1f43b7e41b94cfe7d06", "__v": 0, "code": 6, "flight": { ... }, "price": 2200, "quantity": 8, "type": "FIRST CLASS" }]</pre>	

/flights/:flightId/passengers	GET
-------------------------------	-----

Ritorna i passeggeri relativi al volo :flightId

GET	backend:3000/api/flights/6530a1f43b7e41b94cfe5d05/passengers
-----	--

Gli utenti loggati vedono solo i posti occupati.

<pre>[{ "_id": "6570a1f43b7e41b94cfe9d06", "seat": "F6" }, { "_id": "6570a1f43b7e41b94cfe9d07", "seat": "F6" }]</pre>

mentre l'admin e la compagnia aerea relativa al volo possono conoscere tutti i dati

```
[
  {
    "_id": "6570a1f43b7e41b94cfe9d06",
    "__v": 0,
    "extra": [],
    "name": "Patrick",
    "passportNumber": "IE7654321",
    "purchase": { ...
  },
  "seat": "F6",
  "surname": "O'Connor"
},
{
  "_id": "6570a1f43b7e41b94cfe9d07",
  "__v": 0,
  "extra": [],
  "name": "Sofia",
  "passportNumber": "ES9988776",
  "purchase": { ...
},
  "seat": "F7",
  "surname": "Martinez"
}
```

/flights	POST
----------	------

Crea un nuovo volo, specificando tutti i campi.
Controlla inoltre che departure <= arrival

Admin o compagnia aerea

/flights/:id	DELETE
--------------	--------

Elimina un volo identificato da :id.

Admin o compagnia aerea relativa al volo

/flights/:id	PUT
--------------	-----

Aggiorna un volo identificato da :id. Possono essere specificati tutti i campi.
Controlla che departure <= arrival

Admin o compagnia aerea relativa al volo

Ticket	
code	number
type	'ECONOMY' 'BUSINESS' 'FIRST CLASS'
price	number
quantity	number
flight	ID ['flight']

/tickets	GET
----------	-----

Params: type, from, to, minPrice, maxPrice, minQuantity, maxQuantity, sortBy, order

SortBy: price, quantity, type, code

Order: asc, desc

Ritorna una lista di tutti i biglietti. La ricerca può essere effettuata da un aeroporto di partenza a uno di destinazione come nella ricerca per la rotta. Si possono inoltre filtrare i biglietti in base al prezzo o la quantità disponibile e ordinare i risultati.

GET backend:3000/api/tickets?maxPrice=1000&sortBy=price&order=desc

```
[
  {
    "_id": "6550a1f43b7e41b94cfe7d05",
    "__v": 0,
    "code": 5,
    "flight": { ... },
    "price": 1000,
    "quantity": 25,
    "type": "BUSINESS"
  },
  {
    "_id": "6550a1f43b7e41b94cfe7d0b",
    "__v": 0,
    "code": 11,
    "flight": { ... },
    "price": 600,
    "quantity": 8,
    "type": "ECONOMY"
  },
  {
    "_id": "6550a1f43b7e41b94cfe7d07",
    "__v": 0,
    "code": 7,
    "flight": { ... },
    "price": 550,
    "quantity": 55,
    "type": "ECONOMY"
  },
]
```

/tickets/:id	GET
--------------	-----

Ritorna un biglietto identificato da :id

/tickets	POST
----------	------

Crea un nuovo biglietto, specificando tutti i campi. Verifica che la compagnia aerea che sta creando il biglietto sia la stessa del volo specificato nel biglietto.

Admin o compagnia aerea relativa al volo

/tickets/:id	DELETE
--------------	--------

Elimina un biglietto identificato da :id.

Admin o compagnia aerea relativa al volo

/tickets/:id	PUT
--------------	-----

Aggiorna un biglietto identificato da :id. Possono essere specificati tutti i campi

Admin o compagnia aerea relativa al volo

Purchase	
price	number > 0
date	date, default: now()
quantity	integer > 1
user	ID ['User']
ticket	ID ['Ticket']

/purchases	GET
------------	-----

Params: sortBy, order

sortBy: price, date, quantity

Ritorna una lista di tutti gli acquisti.

Solo admin

GET http://localhost:3000/api/purchases

```
[
  {
    "_id": "6560a1f43b7e41b94cfe8d02",
    "__v": 0,
    "date": "2024-08-02T09:00:00.000Z",
    "price": 400,
    "quantity": 1,
    "ticket": "6550a1f43b7e41b94cfe7d04",
    "user": "650f0a4f3b7e41b94cfe4d10"
  },
  {
    "_id": "6560a1f43b7e41b94cfe8d09",
    "__v": 0,
    "date": "2024-08-09T10:20:00.000Z",
    "price": 450,
    "quantity": 1,
    "ticket": "6550a1f43b7e41b94cfe7d09",
    "user": "650f0a4f3b7e41b94cfe4d10"
  },
  {
  }
```

/purchases/:id

GET

Ritorna un acquisto identificato da :id

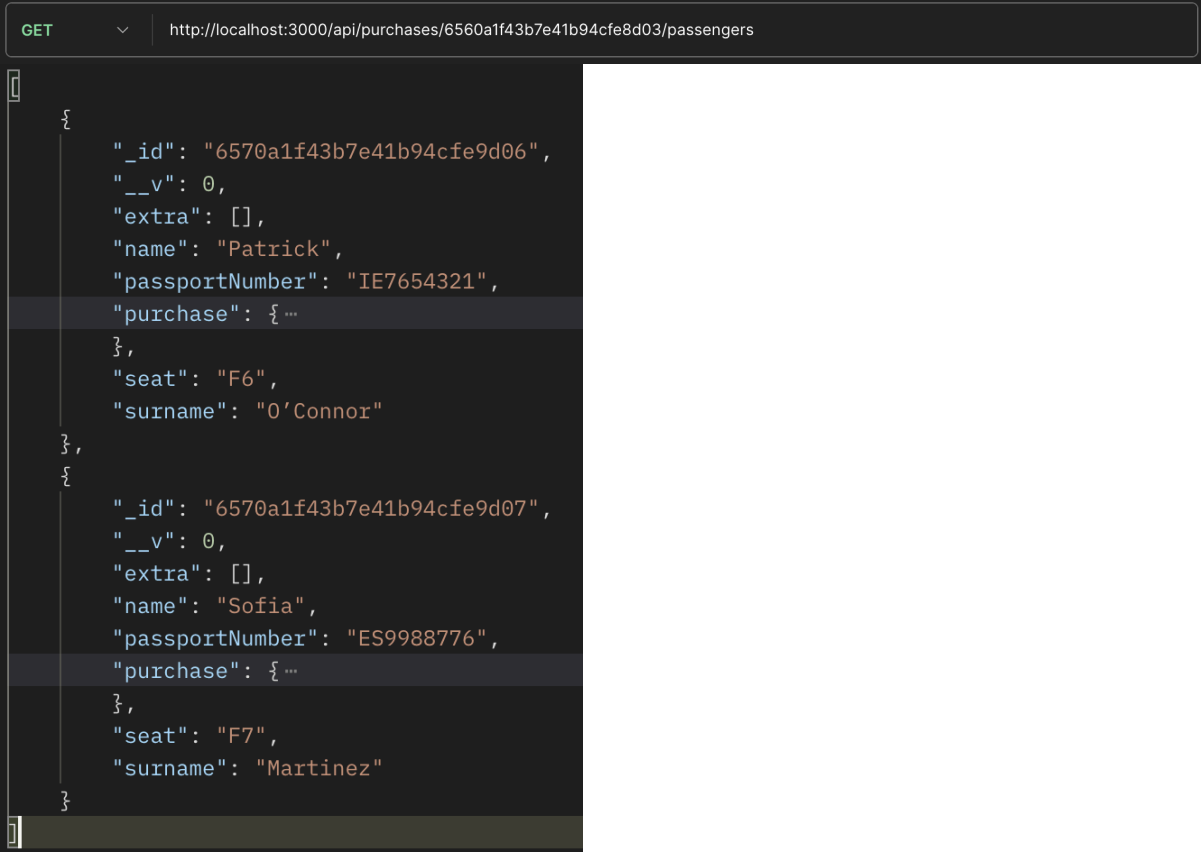
Solo admin

/purchases/:purchaseId/passengers

GET

Ritorna i passeggeri di un acquisto identificato da :id

Solo admin o utente che ha effettuato l'acquisto



```
GET http://localhost:3000/api/purchases/6560a1f43b7e41b94cfe8d03/passengers

{
  "_id": "6570a1f43b7e41b94cfe9d06",
  "__v": 0,
  "extra": [],
  "name": "Patrick",
  "passportNumber": "IE7654321",
  "purchase": { ... },
  "seat": "F6",
  "surname": "O'Connor"
},
{
  "_id": "6570a1f43b7e41b94cfe9d07",
  "__v": 0,
  "extra": [],
  "name": "Sofia",
  "passportNumber": "ES9988776",
  "purchase": { ... },
  "seat": "F7",
  "surname": "Martinez"
}
```

/purchases

POST

Crea un nuovo acquisto, specificando quantity, user e ticket.

Un utente può effettuare un acquisto solo per se stesso

Admin o user

/purchases/:id	DELETE
----------------	--------

Elimina un acquisto identificato da :id, rimborsa l'utente e ripristina la quantità di biglietti disponibile.

E' prima necessario eliminare eventuali passeggeri creati e legati all'acquisto

Admin o user che ha effettuato l'acquisto

/purchases/:id	PUT
----------------	-----

Aggiorna un acquisto identificato da :id. Si possono specificare tutti i campi.

Se un utente vuole aggiornare la quantità di biglietti acquistati dovrà effettuare un nuovo acquisto, in quanto il prezzo del biglietto potrebbe essere cambiato.

Solo admin

Passenger	
name	string
surname	string
CF?	string
passportNumber?	string
extra?	string[] of 'LARGER SEAT' 'PRIORITY' 'EXTRA BAG'
seat	string, /^[A-Z]([1-9]\d*)\$/
purchase	ID ['Purchase']

/passengers	GET
-------------	-----

Params: CF, passportNumber, name, surname, seat, sort by, order

sortBy: name, surname, seat, CF, passportNumber

Ritorna una lista di tutti i passeggeri.

Solo admin

```
GET http://localhost:3000/api/passengers

[
  {
    "_id": "6570a1f43b7e41b94cfe9d01",
    "CF": "LCABNC90A01H501Z",
    "_v": 0,
    "extra": [
      "PRIORITY"
    ],
    "name": "Luca",
    "purchase": { ... },
    "seat": "A1",
    "surname": "Bianchi"
  },
  {
    "_id": "6570a1f43b7e41b94cfe9d09",
    "_v": 0,
    "extra": [
      "EXTRA BAG"
    ],
    "name": "Sophie",
    "passportNumber": "GB1230987",
    "purchase": { ... },
    "seat": "A7",
    "surname": "Smith"
  }
]
```

/passengers/:id	GET
-----------------	-----

Ritorna un passeggero identificato da :id

Solo admin

/passengers	POST
-------------	------

Crea un nuovo passeggero, specificando: name, surname, seat, purchase, uno tra CF e passportNumber ed eventuali extra.

Controlla che il numero di passeggeri già creati per l'acquisto specificato sia minore del numero di biglietti acquistati (purchase.quantity).

Controlla inoltre che il posto esista sull'aeroplano e sia libero.

Admin e utente proprietario dell'acquisto

/passengers/:id	DELETE
-----------------	--------

Elimina un passeggero identificato da :id.

Admin, utente proprietario dell'acquisto e compagnia aerea relativa al volo

/passengers/:id	PUT
-----------------	-----

Aggiorna un passeggero identificato da :id.

Si possono specificare i campi: name, surname, CF, passportNumber, extra, seat.

Se viene specificato un nuovo posto vengono eseguiti gli stessi controlli fatti per la creazione.

Admin, utente proprietario dell'acquisto e compagnia aerea relativa al volo

Itinerary

/itineraries

GET

Params: from, to, fromDate, toDate, onlyDirect, maxStops, airline, sortBy, order

airline: Filtra gli itinerari con almeno un volo di una compagnia specifica (con ricerca simile a LIKE)

sortBy: departure, arrival, duration, numStops

Ritorna tutti gli itinerari da un aeroporto from a un aeroporto to con possibilità di voli diretti e con 1 o 2 scali.

```
GET http://localhost:3000/api/itineraries?from=vce&to=ber

{
  "_id": "6530a1f43b7e41b94cfe5d05",
  "__v": 0,
  "airline": { ... },
  "airplane": { ... },
  "arrival": "2025-09-16T18:30:00.000Z",
  "code": "FR505",
  "departure": "2025-09-16T09:30:00.000Z",
  "duration": 540,
  "route": { ... },
  "stop1": { ... },
  "stop2": { ... },
  "numStops": 2,
  "finalArrival": "2025-09-18T21:15:00.000Z",
  "totDuration": 1980
},
{ ... },
{ ... }
```

Authentication

JWT

1. Creazione della chiave segreta

- All'avvio, il backend controlla se esiste un file .env
- Se non esiste, lo crea e genera una variabile TOKEN_SECRET, una stringa casuale di 128 caratteri

```
const token = crypto.randomBytes(64).toString("hex");
```

2. Generazione JWT

- In fase di login/signup, il backend crea un JWT contenente informazioni rilevanti per l'utente
 - User: id, mail, isAdmin
 - Airline: id, mail, isAirline
- Il token viene firmato con TOKEN_SECRET e scade dopo 7 giorni
- Infine viene inviato al frontend, che potrà utilizzarlo per autenticare le richieste successive, senza la necessità di fare nuovamente il login

```
function generateAccessToken(user) {  
  return jwt.sign(user, process.env.TOKEN_SECRET, { expiresIn: "7d" });  
}
```

3. Middlewares

Vengono inoltre utilizzati dei middleware per intercettare le richieste HTTP che richiedono l'autenticazione dell'utente per:

- Verificare la validità del token
- Verificare che l'utente sia un admin, una compagnia aerea o uno specifico utente o compagnia

Ad esempio una richiesta GET alla risorsa /users è riservata ai soli admin, mentre una POST a /airports solo ad admin e compagnie aeree

```
function authenticateToken(req, res, next) {
  const token = req.headers["authorization"];

  if (token === null) return res.sendStatus(401); // TOKEN MISSING

  jwt.verify(token, process.env.TOKEN_SECRET, (err, user) => {

    if(err && err.name === "TokenExpiredError"){
      return res.status(401).json({error: "Token expired"});
    }

    if (err){
      return res.sendStatus(403); // NOT ALLOWED
    }

    req.user = user;

    next();
  });
}
```

```
function checkAdmin(req, res, next) {
  if (!req.user?.isAdmin) {
    return res.sendStatus(403);
  }

  next();
}
```